4. Data Management

Surafel Luleseged (PhD)

AASTU

March 29, 2026

Content

1. Data preprocessing
2. Discussion

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

1. Data loading / reading

📂 Data Type	📚 Library / Method	📄 Example Code	✅ Use Case
CSV File	pandas	`pd.read_csv("data.csv")`	Most common dataset format
Excel File	pandas	`pd.read_excel("data.xlsx")`	Business/data reports
Text File	Built-in `open()`	`open("file.txt").read()`	Logs, raw text
JSON File	`json` / pandas	`json.load(f)` / `pd.read_json()`	APIs, structured data
SQL Database	`sqlite3` + pandas	`pd.read_sql_query()`	Databases
URL (Online Data)	pandas	`pd.read_csv(url)`	Web datasets
Large Files	pandas	`pd.read_csv(..., chunksize=)`	Big data handling

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.
     - You can remove rows or columns that contain missing values using the dropna() function.

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.
     - You can remove rows or columns that contain missing values using the dropna() function.
     - This is suitable when you have a large dataset and the number of missing values is small, so data loss is minimal.

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.
     - You can remove rows or columns that contain missing values using the dropna() function.
     - This is suitable when you have a large dataset and the number of missing values is small, so data loss is minimal.
     - Remove rows with any missing values:
       df_cleaned = df.dropna()

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.
     - You can remove rows or columns that contain missing values using the dropna() function.
     - This is suitable when you have a large dataset and the number of missing values is small, so data loss is minimal.
     - Remove rows with any missing values:
       df_cleaned = df.dropna()
     - Remove columns with any missing values:
       df_cleaned = df.dropna(axis = columns)

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.
     - You can remove rows or columns that contain missing values using the dropna() function.
     - This is suitable when you have a large dataset and the number of missing values is small, so data loss is minimal.
     - Remove rows with any missing values:
       df_cleaned = df.dropna()
     - Remove columns with any missing values:
       df_cleaned = df.dropna(axis = columns)
     - Remove rows only if all values are missing in that row:
       df_cleaned = df.dropna(how = all)

Data Preprocessing

Data preprocessing is the step in a data workflow where raw data is cleaned and transformed into a usable format for analysis or machine learning.

2. Data Cleaning: Fixing errors and inconsistencies
   - Handling missing values (remove, fill with mean/median, etc.)
   - In Python, missing values (represented as NaN or None in pandas dataframes) can be handled using two primary approaches: deleting them or imputing (filling) them.
     - You can remove rows or columns that contain missing values using the dropna() function.
     - This is suitable when you have a large dataset and the number of missing values is small, so data loss is minimal.
     - Remove rows with any missing values:
       df_cleaned = df.dropna()
     - Remove columns with any missing values:
       df_cleaned = df.dropna(axis = columns)
     - Remove rows only if all values are missing in that row:
       df_cleaned = df.dropna(how = all)
     - Remove rows with missing values in specific columns (e.g., 'age' and 'income'):
       df_cleaned = df.dropna(subset = [age, income])

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0):
       df_filled = df .fillna(0)

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0): df_filled = df .fillna(0)
     - Fill with an aggregate statistic (mean, median, or mode) of the column

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0): df_filled = df .fillna(0)
     - Fill with an aggregate statistic (mean, median, or mode) of the column
     - Fill with the previous or next value (for time-series or ordered data) (backward forward fill)

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0): df_filled = df .fillna(0)
     - Fill with an aggregate statistic (mean, median, or mode) of the column
     - Fill with the previous or next value (for time-series or ordered data) (backward forward fill)
     - Interpolate missing values: Estimates values based on neighboring data points, which is especially useful for time series.

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0): df_filled = df .fillna(0)
     - Fill with an aggregate statistic (mean, median, or mode) of the column
     - Fill with the previous or next value (for time-series or ordered data) (backward forward fill)
     - Interpolate missing values: Estimates values based on neighboring data points, which is especially useful for time series.
   - Removing duplicates: In Python, the most common methods to remove duplicates from a list are using set() (if order doesn't matter) or dict.fromkeys() (if order must be preserved).

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0): df_filled = df .fillna(0)
     - Fill with an aggregate statistic (mean, median, or mode) of the column
     - Fill with the previous or next value (for time-series or ordered data) (backward forward fill)
     - Interpolate missing values: Estimates values based on neighboring data points, which is especially useful for time series.
   - Removing duplicates: In Python, the most common methods to remove duplicates from a list are using set() (if order doesn't matter) or dict.fromkeys() (if order must be preserved).
     - Remove duplicate rows (all columns considered by default): df_cleaned = df.dropduplicates()

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Handling missing values
     - Instead of deleting data, you can replace missing values with an appropriate substitute. The choice of the replacement value depends on the nature of your data.
     - Fill with a static constant value (e.g., 0): df_filled = df .fillna(0)
     - Fill with an aggregate statistic (mean, median, or mode) of the column
     - Fill with the previous or next value (for time-series or ordered data) (backward forward fill)
     - Interpolate missing values: Estimates values based on neighboring data points, which is especially useful for time series.
   - Removing duplicates: In Python, the most common methods to remove duplicates from a list are using set() (if order doesn't matter) or dict.fromkeys() (if order must be preserved).
     - Remove duplicate rows (all columns considered by default): df_cleaned = df.dropduplicates()
     - Remove duplicates based on specific columns: df_unique_names = df.drop_duplicates(subset = [Name])

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Correcting incorrect data types (e.g., text instead of numbers)

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Correcting incorrect data types (e.g., text instead of numbers)
     - Convert an entire column to a specific type:
       df [column name] = df [column name].astype(int)
       df [another column] = df [another column].astype(str)
       df [float column] = df [float column].astype(float)

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Correcting incorrect data types (e.g., text instead of numbers)
     - Convert an entire column to a specific type:
       df [column name] = df [column name].astype(int)
       df [another column] = df [another column].astype(str)
       df [float column] = df [float column].astype(float)
     - Convert to numeric, handling errors: When a column that should be numeric contains non-numeric strings (e.g., "N/A", "$400", "Unknown") that would cause a ValueError with astype(), the pd.to numeric() function is more robust
       df [age] = pd.tonumeric(df [age],errors = coerce)

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Correcting incorrect data types (e.g., text instead of numbers)
     - Convert an entire column to a specific type:
       df [column name] = df [column name].astype(int)
       df [another column] = df [another column].astype(str)
       df [float column] = df [float column].astype(float)
     - Convert to numeric, handling errors: When a column that should be numeric contains non-numeric strings (e.g., "N/A", "$400", "Unknown") that would cause a ValueError with astype(), the pd.to numeric() function is more robust
       df [age] = pd.tonumeric(df [age],errors = coerce)
     - The errors='coerce' argument turns any value it cannot convert into a numeric type into NaN.

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Fixing outliers or invalid values

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Fixing outliers or invalid values
     - Removal (Trimming): Deleting rows containing outliers if they are errors or insignificant.

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Fixing outliers or invalid values
     - Removal (Trimming): Deleting rows containing outliers if they are errors or insignificant.
     - Capping/Flooring (Winsorizing): Replacing extreme values with specific percentiles (e.g., 5th or 95th) or the upper/lower bounds calculated by IQR.

Data Preprocessing

2. Data Cleaning: Fixing errors and inconsistencies (cont'd)
   - Fixing outliers or invalid values
     - Removal (Trimming): Deleting rows containing outliers if they are errors or insignificant.
     - Capping/Flooring (Winsorizing): Replacing extreme values with specific percentiles (e.g., 5th or 95th) or the upper/lower bounds calculated by IQR.
     - Imputation: Replacing outliers with more central values like the mean, median, or mode.

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices
     - The join() method is a convenient shortcut that defaults to merging on indices.

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices
     - The join() method is a convenient shortcut that defaults to merging on indices.
     - The pd.concat() function is primarily used for stacking DataFrames along an axis, either vertically (adding rows, axis=0) or horizontally (adding columns, axis=1), without looking for matching key values.

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices
     - The join() method is a convenient shortcut that defaults to merging on indices.
     - The pd.concat() function is primarily used for stacking DataFrames along an axis, either vertically (adding rows, axis=0) or horizontally (adding columns, axis=1), without looking for matching key values.
   - Normalization (scaling values between 0 and 1)($X_N = \frac{X - X_{min}}{X_{max} - X_{min}}$)

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices
     - The join() method is a convenient shortcut that defaults to merging on indices.
     - The pd.concat() function is primarily used for stacking DataFrames along an axis, either vertically (adding rows, axis=0) or horizontally (adding columns, axis=1), without looking for matching key values.
   - Normalization (scaling values between 0 and 1)($X_N = \frac{X - X_{min}}{X_{max} - X_{min}}$)
   - Standardization (mean = 0, standard deviation = 1)

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices
     - The join() method is a convenient shortcut that defaults to merging on indices.
     - The pd.concat() function is primarily used for stacking DataFrames along an axis, either vertically (adding rows, axis=0) or horizontally (adding columns, axis=1), without looking for matching key values.
   - Normalization (scaling values between 0 and 1)($X_N = \frac{X - X_{min}}{X_{max} - X_{min}}$)
   - Standardization (mean = 0, standard deviation = 1)
   - Robust scaling ($X_{scaled} = \frac{Xmedian}{IQR}$)

Data Preprocessing

3. Data Integration (Combining data from multiple sources) and Data Transformation (Changing data into a more useful format)
   - Merging tables and Joining datasets
     - The pd.merge() function is used to combine DataFrames based on common columns (keys) or indices
     - The join() method is a convenient shortcut that defaults to merging on indices.
     - The pd.concat() function is primarily used for stacking DataFrames along an axis, either vertically (adding rows, axis=0) or horizontally (adding columns, axis=1), without looking for matching key values.
   - Normalization (scaling values between 0 and 1)($X_N = \frac{X - X_{min}}{X_{max} - X_{min}}$)
   - Standardization (mean = 0, standard deviation = 1)
   - Robust scaling ($X_{scaled} = \frac{Xmedian}{IQR}$)
   - Encoding categorical variables (e.g., converting "Male/Female" to 0/1)

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables
   - Combining columns (e.g., total price = quantity $\times$ unit price)

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables
   - Combining columns (e.g., total price = quantity $\times$ unit price)
   - Extracting parts of dates (year, month, day)

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables
   - Combining columns (e.g., total price = quantity $\times$ unit price)
   - Extracting parts of dates (year, month, day)
6. Data Splitting: Preparing data for modeling:

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables
   - Combining columns (e.g., total price = quantity $\times$ unit price)
   - Extracting parts of dates (year, month, day)
6. Data Splitting: Preparing data for modeling:
   - Training set (to train the model)

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables
   - Combining columns (e.g., total price = quantity $\times$ unit price)
   - Extracting parts of dates (year, month, day)
6. Data Splitting: Preparing data for modeling:
   - Training set (to train the model)
   - Test set (to evaluate performance)

Data Preprocessing

4. Data Reduction: Reducing dataset size while keeping important information
   - Feature selection (choosing only useful columns)
   - Dimensionality reduction (e.g., Principal Component Analysis (PCA))
   - Sampling
5. Feature Engineering: Creating new meaningful variables
   - Combining columns (e.g., total price = quantity $\times$ unit price)
   - Extracting parts of dates (year, month, day)
6. Data Splitting: Preparing data for modeling:
   - Training set (to train the model)
   - Test set (to evaluate performance)
   - Sometimes validation set

Transportation data and data soirces

- Discussion